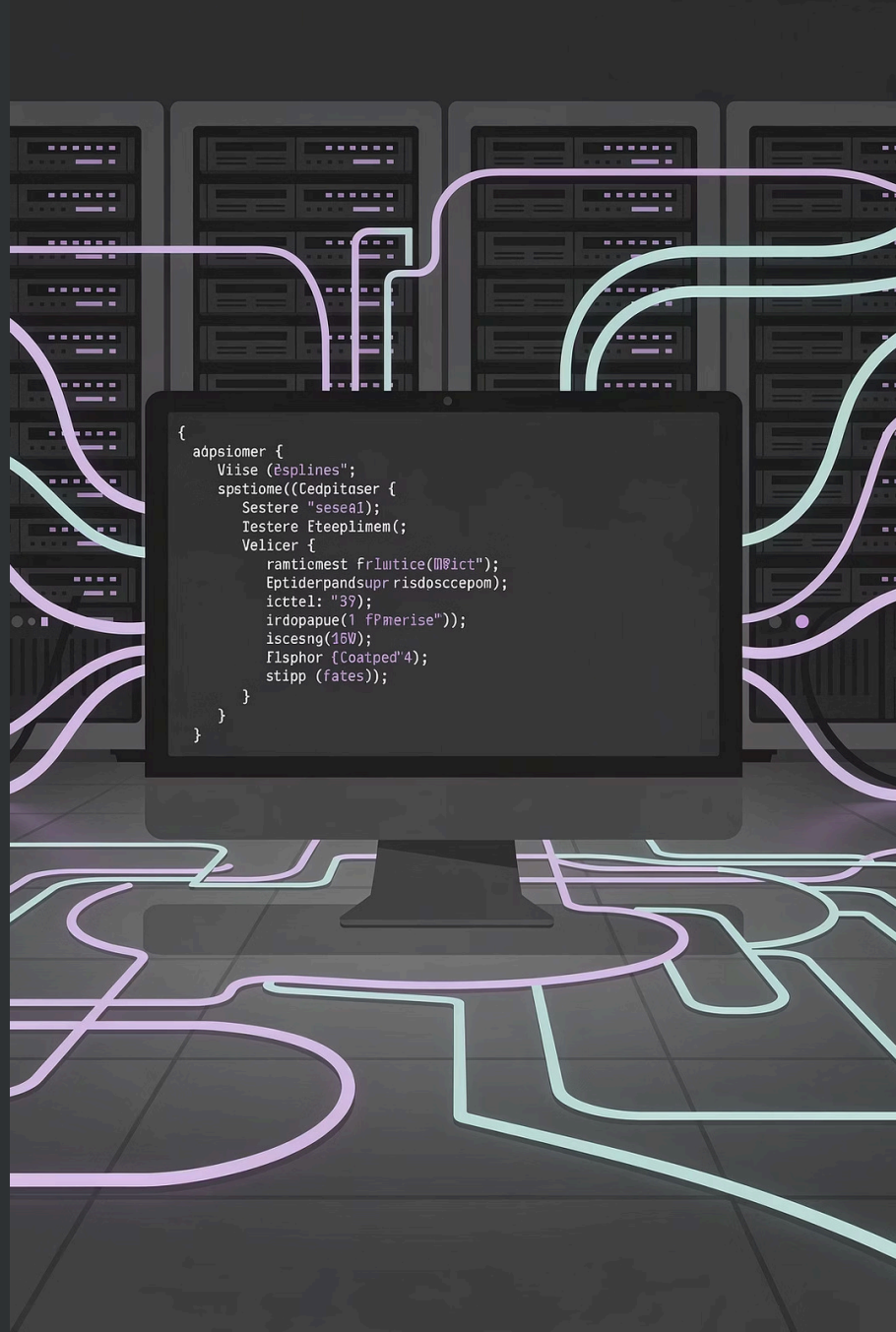
Python Back-End: O Despertar da Lógica

Construindo as bases sólidas para sistemas escaláveis.

UNIDADE CURRICULAR 3

BACK-END



O que é o Back-End?

Antes de escrever código, vamos entender onde você está entrando. Pense em um restaurante cheio de movimento:

🍽️ Front-End

O salão do restaurante — a parte que o cliente vê, toca e interage. É bonito, responsivo e agradável.

🍳 Back-End

A cozinha — onde a mágica acontece. As regras de negócio são aplicadas, os dados são processados e as respostas são preparadas.

🛎️ A API / HTTP

O garçom — leva o pedido da mesa até a cozinha e traz a resposta de volta. Sem ele, front e back não se falam.

Por que revisar Lógica antes de POO?

Muita gente quer pular direto para frameworks como **Django** ou **FastAPI** — e acaba se perdendo. A razão é simples: frameworks são construídos em cima de peças básicas da linguagem. Se a sua lógica vacila, o servidor cai.

Programação Orientada a Objetos (POO) não é um bicho de sete cabeças: ela é apenas uma forma elegante de **organizar a lógica que você já conhece**. Antes de organizar, precisa dominar.

1. Frameworks = peças básicas
2. Lógica fraca = servidor que cai
3. POO = lógica organizada

Condicionais: As Regras de Negócio

O servidor é, no fundo, um grande tomador de decisões. A cada requisição que chega, ele precisa avaliar condições e escolher o caminho certo. É exatamente o que o `if/elif/else` faz.

"O usuário está autenticado?"

Se não, retorna 401 Unauthorized.

"O produto tem estoque?"

Se não, retorna 422 Unprocessable.

"O pagamento foi aprovado?"

Se sim, cria o pedido no banco.

```
status_pagamento = "aprovado"
tem_estoque = True

if not tem_estoque:
 print("Erro: sem estoque")
elif status_pagamento != "aprovado":
 print("Erro: pagamento recusado")
else:
 print("Pedido criado com sucesso!")
```

Prática 1: Sistema de Login

🎯 O Desafio

Simule um sistema de login verificando se o usuário e senha fornecidos batem com um "banco de dados" fictício armazenado em variáveis.

```
db_usuario = "admin"
db_senha   = "1234"

usuario = input("Usuário: ")
senha   = input("Senha: ")

if usuario == db_usuario and senha
== db_senha:
    print("200 OK — Bem-vindo!")
else:
    print("401 Unauthorized")
```

📡 Status Codes em Ação

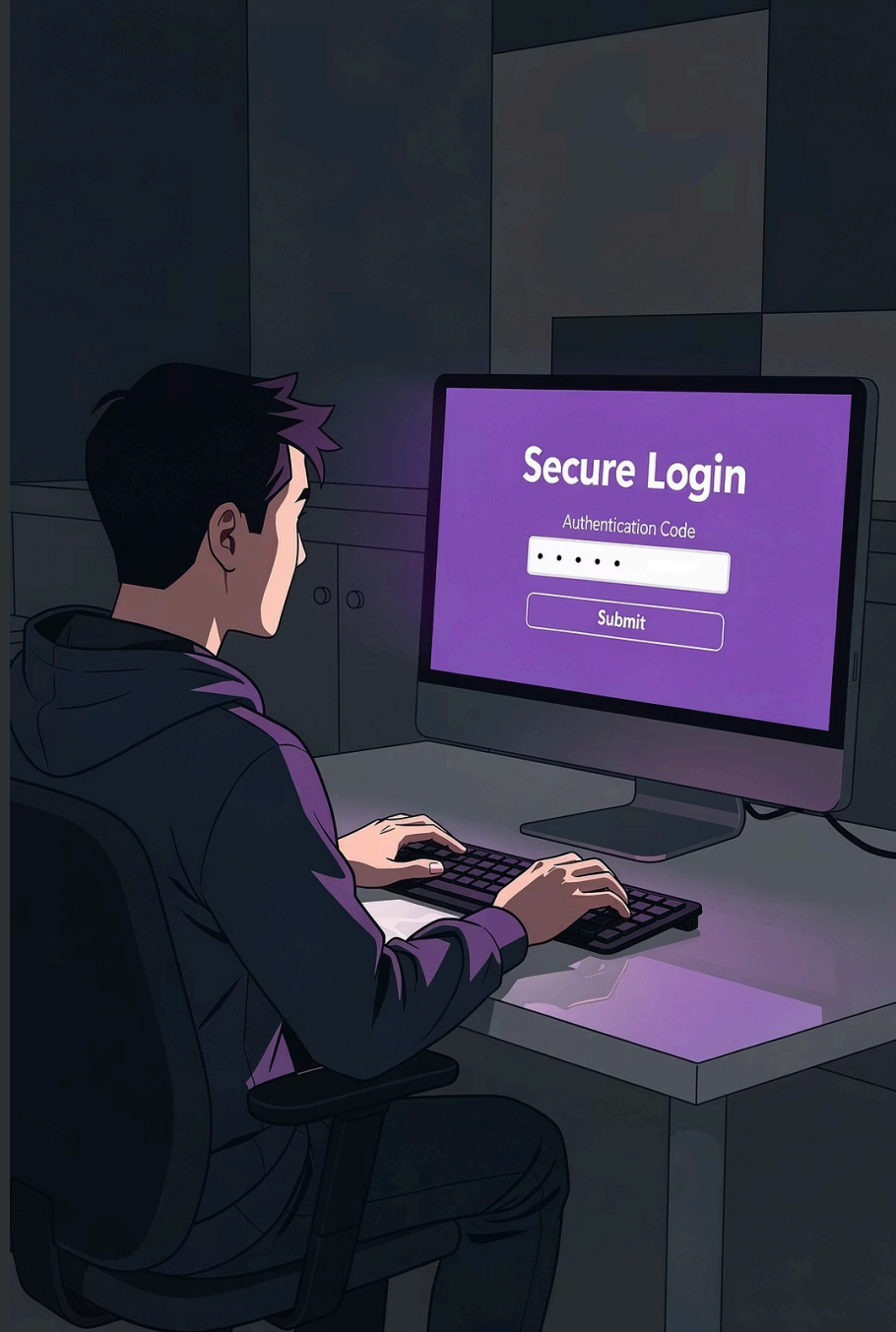
200 OK

Login realizado com sucesso. Sessão iniciada.

401 Unauthorized

Credenciais inválidas. Acesso negado ao recurso.

Perceba: as respostas HTTP não são reinventadas a cada projeto. Elas seguem um padrão universal que todo back-end respeita.



Dicionários e Listas: O Coração dos Dados

No back-end, quase toda informação trafega no formato **JSON**. E adivinhe? Em Python, JSON é exatamente um dicionário ou uma lista. Dominar essas estruturas é dominar 80% da manipulação de dados.

Modelando um Usuário

```
usuario = {
    "id": 1,
    "nome": "Ana Silva",
    "email": "ana@email.com",
    "ativo": True
}
```

Lista de Produtos

```
produtos = [
    {"id": 1, "nome": "Notebook", "preco": 3500},
    {"id": 2, "nome": "Mouse",    "preco": 80},
]
```

Dict = Registro

Cada dicionário representa um objeto do mundo real: usuário, produto, pedido, transação.

List = Coleção

Listas agrupam vários registros, como as linhas de uma tabela de banco de dados.

Laços de Repetição: Processamento em Massa

Imagine que seu banco de dados retornou **mil registros**. Você não vai processar um por um na mão — é para isso que existe o `for`. No back-end, laços são usados o tempo todo para tarefas repetitivas e escaláveis.

1

Filtrar Dados

Percorrer registros e selecionar apenas os que atendem a uma condição específica.

2

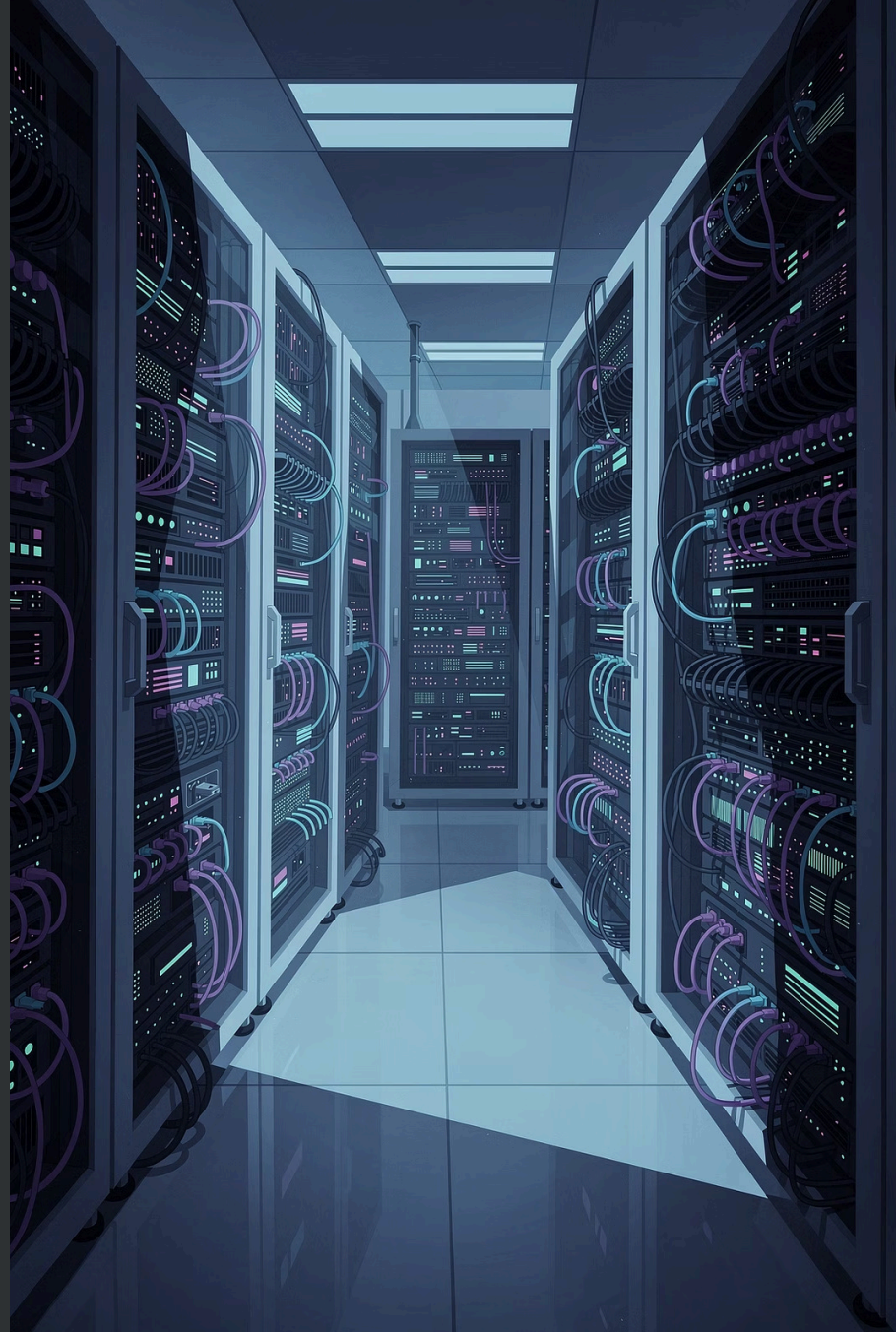
Calcular Totais

Somar vendas, acumular pontos, calcular médias de avaliações de produtos.

3

Enviar em Massa

Disparar e-mails, notificações push ou relatórios para todos os usuários de uma lista.



Prática 2: Filtro de Usuários Ativos

💻 O Código

```
usuarios = [
 {"nome": "Ana", "ativo": True},
 {"nome": "Bruno", "ativo": False},
 {"nome": "Carla", "ativo": True},
 {"nome": "Daniel", "ativo": False},
 {"nome": "Eva", "ativo": True},
]

print("Usuários ativos:")
for u in usuarios:
 if u["ativo"]:
 print(f"
```

Funções: Modularização e o Princípio DRY

DRY = Don't Repeat Yourself. No back-end, código duplicado é um pesadelo: se há um bug, você precisa corrigir em dez lugares. Funções são seus trabalhadores especializados — cada um sabe fazer uma coisa muito bem.

1. **Input (Request)**
   A função recebe os dados da requisição como parâmetros.
2. **Processamento**
   Aplica as regras de negócio: valida, calcula, consulta.
3. **Output (Response)**
   Retorna o resultado com `return` — equivalente à resposta HTTP.

Prática 3: Função de Busca (Error 404)

💻 Simulando uma Rota de API

```
usuarios_db = {
    1: "Ana Silva",
    2: "Bruno Costa",
    3: "Carla Lima",
}

def buscar_usuario(id):
    if id in usuarios_db:
        return f"200 OK — {usuarios_db[id]}"
    else:
        return "404 Not Found — Usuário inexistente"

print(buscar_usuario(2))
print(buscar_usuario(99))
```

🌐 O que você acabou de simular

GET /usuarios/2

200 OK → Retorna os dados do usuário encontrado.

GET /usuarios/99

404 Not Found → Recurso não existe no servidor.

Esta é exatamente a lógica por trás de qualquer rota de busca em Django, FastAPI ou Flask.

O Próximo Passo: De Funções para Classes

O problema das funções soltas

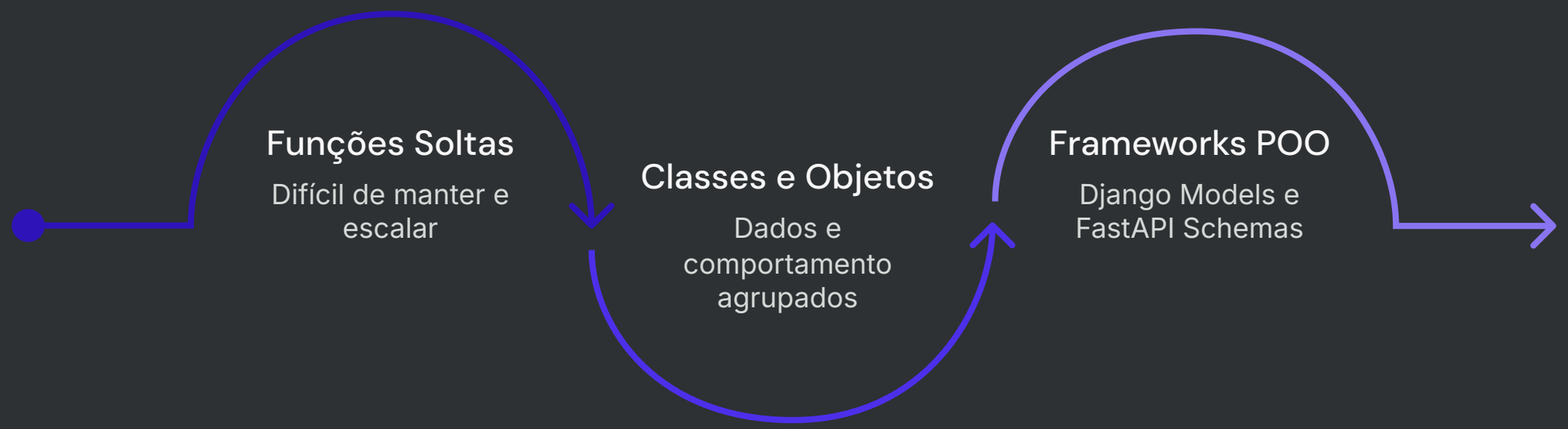
Conforme o projeto cresce, funções sem organização se tornam um caos. Imagine ter `buscar_usuario()`, `atualizar_usuario()`, `deletar_usuario()` espalhadas pelo código sem nenhuma relação explícita entre elas.

> Isso se chama **código espaguete** — difícil de manter, testar e escalar.

A solução: Objetos

Um **Objeto** reúne **dados + funções** que pertencem ao mesmo contexto. Na próxima aula, vamos transformar nossos dicionários de usuário em **Classes Python** — o coração da POO.

Spoiler: tudo que você aprendeu hoje *continua valendo*. A POO só organiza melhor o que já sabemos fazer.



Conclusão e Próximos Passos

Você acabou de revisitar as fundações que sustentam qualquer servidor Python. Não são apenas conceitos acadêmicos — são as engrenagens rodando por baixo de todo grande sistema.

Condicionais

As regras de negócio que decidem cada resposta do servidor.

Dicionários e Listas

A forma Python de modelar e transportar dados em JSON.

Laços de Repetição

Processamento em massa de registros vindos do banco de dados.

Funções

Modularização e o princípio DRY para código limpo e reutilizável.

> "O back-end é onde a inteligência do software reside. Dominar sua lógica é dominar o sistema inteiro."

📌 Próxima aula: **Programação Orientada a Objetos** — transformando tudo isso em Classes, Atributos e Métodos. Dúvidas? Agora é a hora! 🙋